

Sincronizar schema: o que a tarefa 4.1 não consegue medir

A tarefa pede pra conferir se as migrations 004–007 foram aplicadas em gc-dev e gc-prod. Existem migrations até a 010, não há registro de nada do que já rodou, e os dois critérios de aceite passam hoje — com os types desatualizados.

REPOSITÓRIO	BRANCH	COMMIT LIDO
cremalet/gc-sistema	feature-dev-breno-automacao	e2dcc57
DATA		
2026-09-01		

ESCOPO

Toda a análise vem da leitura do repositório. Nada foi executado contra gc-dev ou gc-prod — não há credenciais versionadas (`.env.local` fica fora do repo) nem projeto linkado pela CLI. Quem diz o que está aplicado é o script de verificação, rodado por quem tem acesso aos dois painéis.

Os cinco problemas, do mais grave pro menos

#	GRAVIDADE	PROBLEMA	IMPACTO
1	CRÍTICO	004 é destrutiva, não idempotente e sensível à ordem	Aplicada em prod sem backup/janela, dropa 13 colunas de obras com dados. Fora de ordem, reverte a 007 e derruba /obras .
2	CRÍTICO	Escopo desatualizado: são 004-010, não 004-007	008, 009 e 010 entraram depois da tarefa ser criada. A 010 é pré-requisito da ingestão via WhatsApp em prod.
3	ALTO	Não existe controle de migrations	Sem supabase/migrations , sem CLI, sem tabela de controle: "rodar as pendentes" não corresponde a comando nenhum.
4	ALTO	Critérios de aceite são falso-positivos	tsc e lint já passam com os types velhos. Um as unknown as apaga o drift.
5	MÉDIO	types.ts está três objetos atrás do schema	E db:types tem o --project-id fixo: não dá pra gerar os types do outro ambiente.

1 A 004 é destrutiva, não idempotente e sensível à ordem

A `004_revisao_schema.sql` cria a tabela `clientes`, migra os dados de cliente que moravam em `obras`, e então dropa as colunas antigas — sem `if not exists` na criação e sem `if exists` nos drops. Ela está envolvida em `begin/commit`, então uma segunda execução aborta inteira em vez de deixar meio caminho. O risco não é re-rodar: é rodar a primeira vez em produção.

MIGRATION	IDEMPOTENTE	EM TRANSAÇÃO	POR QUÊ
004	não	sim	<code>create table</code> e <code>drop column</code> sem guard; migra dados de <code>obras</code> pra <code>clientes</code>
005	sim	não	<code>drop policy if exists</code> antes de cada <code>create policy</code>
006	sim	n/a	<code>create or replace function</code> , statement único
007	sim	n/a	<code>create or replace view</code> , statement único
008	não	não	<code>create table documentos_processamento</code> sem guard, seguida de índices, trigger e 4 policies
009	não	não	<code>create table contatos_whatsapp</code> sem guard, mesma estrutura
010	sim	não	<code>on conflict do nothing</code> no bucket + <code>drop policy if exists</code>

O que a 004 remove, em definitivo e sem backup: de `obras`, as colunas `cliente`, `contato`, `cpf_cnpj`, `telefone`, `email`, `valor_total`, `desconto`, `valor_final`, os quatro `pct_*`, `forma_pagamento` e `doc_url` — treze no total; de `orcamentos`, outras seis. O cabeçalho do próprio arquivo pede janela de manutenção. O checklist da tarefa diz apenas "aplicar as mesmas migrations em gc-prod": não menciona backup nem janela.

A ordem não é sugestão

004

revisão de schema
recria as 4 views

→ **005**

RLS + policies
de clientes

→ **006**

conserta
calcular_valores_obra

→ **007**

progresso_itens_pct
na view

008

documentos_
processamento

→ **009**

contatos_
whatsapp

→ **010**

bucket + policies
de storage

004 DEPOIS DE 007 REVERTE A 007

A 004 recria `obras_com_valores` no final (linha 536) *sem* `progresso_itens_pct`. A listagem seleciona essa coluna por nome em `src/app/(app)/obras/page.tsx:51` — a página quebra em runtime, não em build.

004 SEM 006 DERRUBA A LISTAGEM INTEIRA

A 004 dropa os campos legacy que a `calcular_valores_obra` assumia, e a view chama a função via `LATERAL`. Aplicar a 004 em prod e parar ali deixa `/obras` fora do ar.

2 O escopo real é 004–010

Três migrations entraram no último commit (`e2dcc57 – migration e documentacao`), depois da tarefa ter sido criada:

```
supabase/004_revisao_schema.sql
supabase/005_rls_clientes.sql
supabase/006_fix_calcular_valores_obra.sql
supabase/007_progresso_itens_obra.sql
supabase/008_documentos_processamento.sql ← e2dcc57
supabase/009_contatos_whatsapp.sql ← e2dcc57
supabase/010_storage_bucket_documentos_processamento.sql ← e2dcc57
```

A 010 é a mais fácil de esquecer e a mais barulhenta quando falta: ela cria o bucket `documentos-processamento` e as policies de `storage.objects` que exigem o `empresa_id` como primeiro segmento do path. O workflow n8n de ingestão via WhatsApp já grava em `{empresa_id}/whatsapp/{messageId}-{fileName}` contando com isso. Se a 010 não estiver em prod, a ingestão falha em prod — e nenhum item do checklist original detecta, porque storage não aparece nos types nem no compilador.

3 Ninguém sabe o que já rodou — e não é descuido, é ausência de mecanismo

Não existe `supabase/migrations/`, não existe `supabase/config.toml`, não existe projeto linkado pela CLI. Os `.sql` são scripts numerados aplicados à mão no SQL Editor. Daí decorrem três coisas:

- não há tabela de controle (`schema_migrations`) pra consultar;
- "rodar as migrations pendentes em gc-dev" não corresponde a comando nenhum;
- a única forma de conferir é **inspecionar o catálogo do Postgres**.

Foi pra isso que escrevi `supabase/verificar_schema_aplicado.sql`: somente leitura, seguro pra rodar em produção. Cola no SQL Editor de cada projeto e sai uma linha `OK` ou `FALTA` por evidência — tabela `clientes`, colunas legacy dropadas, FK `obras_cliente_fk`, as 4 views recriadas, RLS e contagem de policies, o CASO 2 da `calcular_valoresobra`, a coluna `progresso_itens_pct`, as duas tabelas novas, o bucket e suas policies — mais contagens totais de tabelas e policies pra diff direto entre dev e prod.

4 Os critérios de aceite não podem falhar

Rodados nesta branch, agora, com os types desatualizados:

```
$ npx tsc --noEmit
(sem erros)

$ npm run lint
✓ No ESLint warnings or errors
```

Os itens 4 e 5 do checklist já estão verdes *apesar* do drift, por dois motivos concretos: `src/lib/types.ts:232` declara `progresso_itens_pct` à mão, e `src/app/(app)/obras/page.tsx:96` faz `(data ?? [])` as unknown as `ObraListItem[]`. O `as unknown as` apaga qualquer divergência entre a string do `select()` e o `Database` gerado. Enquanto ele existir, o `tsc` nunca vai apontar coluna faltando — nem agora, nem na próxima migration.

5 O drift de types, objeto por objeto

`src/lib/supabase/types.ts` tem 2.267 linhas e não contém nenhum destes três:

<code>MIGRATION 007</code> <code>obras_com_valores</code> <code>.progresso_itens_pct</code> <code>ausente da view tipada</code>	<code>MIGRATION 008</code> <code>documentos_processamento</code> <code>tabela ausente</code>	<code>MIGRATION 009</code> <code>contatos_whatsapp</code> <code>tabela ausente</code>
--	--	---

E o script de geração aponta pra um projeto fixo, sem chave de ambiente:

```
// package.json
"db:types": "npx --yes supabase gen types typescript
               --project-id gzbmhgnpoehormnidmvg > src/lib/supabase/types.ts"
```

O repositório não documenta se `gzbmhgnpoehormnidmvg` é `gc-dev` ou `gc-prod`, e não há como gerar os types do outro ambiente — que é exatamente o que a tarefa pede pra comparar. Caminho: dois scripts (`db:types:dev` / `db:types:prod`) ou `--project-id $SUPABASE_PROJECT_ID`.

Checklist corrigido

Diagnóstico

passos 1-3

- 01 ☐ Rodar `supabase/verificar_schema_aplicado.sql` em `gc-dev` e salvar a saída
- 02 ☐ Rodar o mesmo em `gc-prod` e salvar a saída
- 03 ☐ Diffar as duas saídas — essa é a lista real de pendências por ambiente

Preparação

passos 4-5

- 04 ☐ Confirmar qual projeto é `gzbmhgnpoeormnidm` e ajustar `db:types` pra alcançar os dois
- 05 ☐ Envolver 008 e 009 em `begin; ... commit;`

Aplicação

passos 6-8

- 06 ☐ Aplicar o que falta em `gc-dev`, na ordem 004→010, e revalidar com o script
- 07 ☒ **Backup + janela de manutenção** antes de qualquer coisa em `gc-prod` — a 004 é irreversível
- 08 ☐ Aplicar em `gc-prod` na mesma ordem e revalidar com o script

Fechamento

passos 9-10

- 09 ☐ `npm run db:types` e conferir no diff que os três objetos ausentes apareceram
- 10 ☐ `npm run lint` e `npm run tsc` — sabendo que passam por causa do cast

Fora do bloco, recomendado

passos 11-12

- 11 ☐ Remover o `as unknown as` de `obras/page.tsx` pra que o `tsc` volte a detectar drift de schema
- 12 ☐ Migrar pra `supabase/migrations` + CLI, pra que "o que já rodou" pare de ser uma pergunta sem resposta

O que está entregue

supabase/verificar_schema_aplicado.sql

Uma única query, somente leitura, sobre os catálogos do Postgres. Cobre 004 a 010 com ~20 evidências e devolve `migration · status · checagem · esperado · encontrado` ordenado por `migration`. Seguro em produção: não há DDL nem DML. Validado como Postgres antes de entregar.

docs/4.1-sincronizar-schema-analise.md

Esta análise em markdown, versionada junto do código, com o checklist corrigido.

branch worktree-verificar-schema-4-1 · commit a600f06

O push pro GitHub falhou com 403 — `brenocmlo` não tem permissão de escrita em `cremalet/gc-sistema`. O commit está no `.git` compartilhado do repositório local e sobrevive à remoção do worktree.

Análise por leitura estática do repositório Nenhum comando executado contra `gc-dev` ou `gc-prod`
e2dcc57 · feature-dev-breno-automacao